L4: Intermediate SQL

Md. Tariquzzaman
Junior Lecturer, Dept. of Computer Science and Engineering
Islamic University of Technology (IUT)

CSE 4307: Database Management Systems

Contents

Join Expressions

Views

Transactions

Integrity Constraints

Date and Time

Create Table Extensions

Authorization

Chapter Overview

We continue our study of SQL by examining more complex forms of queries and database management features:

- **Join Expressions**: Natural, Inner, and Outer joins.
- **Views**: Virtual relations and materialized views.
- **Transactions**: Atomic units of work (Commit/Rollback).
- **Integrity Constraints**: Ensuring data consistency.
- **Data Types**: Date/Time, Large Objects, and User-Defined types.
- **Authorization**: Managing privileges and roles.

The Sample Data Used in This Lecture

To understand Joins and Views, we will refer to these two tables throughout the slides.

Table 1: Student

ID	Name	Dept
101	Alice	CS
102	Bob	EE
103	Carol	CS
104	Dave	Bio

Table 2: Takes (Course Enrolment)

ID	Course	Grade
101	CS-101	A
101	CS-102	B
102	EE-200	A
999	Unknown	C

Join Expressions

4.1 Join Expressions: Introduction

Why do we need Joins?

- In previous chapters, we combined relations using the **Cartesian product** (e.g., `student, takes`).
- A Cartesian product pairs *every* row from table A with *every* row from table B.
- This often requires a `WHERE` clause to filter meaningful combinations (e.g., `WHERE student.id = takes.id`).

The Solution:

- **Join operations** allow programmers to write queries in a more natural way.
- They help express logic that is difficult or verbose with just Cartesian products.

4.1.1 The Natural Join (Theory)

The **Natural Join** matches tuples with the same value on *all* attributes that share the same name in both relations.

Behavior:

- It considers only pairs of tuples where common attributes match.
- Common attributes appear only **once** in the result.
- **Implicit:** You do not need to specify the columns; SQL finds them automatically.

SQL Syntax:

```
SELECT name, course
FROM student NATURAL JOIN takes;
```

Danger of Natural Join (The Trap)

Natural join equates **all** common attributes. This can lead to critical errors if schemas share unrelated attribute names.

The Trap Query:

```
SELECT name, title
FROM student
NATURAL JOIN takes
NATURAL JOIN course;
```

Why is this dangerous?

- `student` and `takes` join on `ID`. (Correct)
- `takes` and `course` join on `course_id`. (Correct)
- **BUT:** `student` and `course` BOTH have a `dept_name` column.

Danger of Natural Join (The Consequence)

Because of the shared `dept_name` column:

The Logic Error

The query incorrectly requires the **Student's Department** to be exactly the same as the **Course's Department**.

Result:

- If a CS student takes a Music elective, that row is **excluded**.
- Valid data is hidden because of an unintended join condition.

4.1.1 The USING Clause

To avoid the natural join trap, use `JOIN ... USING`. It specifies exactly which columns to equate.

Corrected Query:

```
SELECT name, title
FROM (student NATURAL JOIN takes)
JOIN course USING (course_id);
```

Why this works:

- This forces the join to match *only* on `course_id`.
- `dept_name` is allowed to differ between student and course, preserving electives.

4.1.2 Join Condition (ON Clause)

The `ON` condition is the most versatile. It allows any general predicate.

Syntax:

```
SELECT name, course, grade
FROM student JOIN takes
ON student.id = takes.id;
```

Result (Inner Join):

Name	Course	Grade
Alice	CS-101	A
Alice	CS-102	B
Bob	EE-200	A

Note: Dave (104) is missing because he has no matching ID in Takes.

4.1.3 Outer Joins: The Concept

Inner Join Limitation:

- Tuples without matches are lost (Data Loss).
- As seen previously, Student "Dave" disappeared.

Outer Join Solution:

- Preserves tuples that would be lost by padding them with `NULL` values.

Three Types:

1. **Left Outer Join**: Preserves tuples from the left relation.
2. **Right Outer Join**: Preserves tuples from the right relation.
3. **Full Outer Join**: Preserves tuples from both relations.

Left Outer Join: Example

Goal: List ALL students, even if they haven't taken a course.

```
SELECT name, course, grade
FROM student LEFT OUTER JOIN takes
ON student.id = takes.id;
```

Result:

Name	Course	Grade
Alice	CS-101	A
Bob	EE-200	A
Dave	**NULL**	**NULL**

Dave is preserved. The missing course info is filled with NULL.

Full Outer Join: Example

Goal: Keep ALL students AND all course records, even unmatched ones (orphaned records).

```
SELECT name, course, grade
FROM student FULL OUTER JOIN takes
ON student.id = takes.id;
```

Result:

Name	Course	Grade
Alice	CS-101	A
Dave	NULL	NULL
NULL	**Unknown**	**C**

ID 999 (Unknown course) is preserved from the right table, Name is NULL.

Left, Right and Full Outer Join

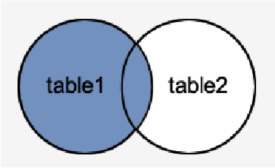


Figure: Left Outer Join

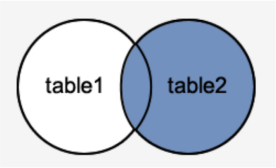


Figure: Right Outer Join

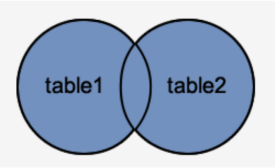


Figure: Full Outer Join

Views

4.2 Views: Overview

Normally, relations are stored physically in the database. SQL allows a **virtual relation** to be defined by a query.

Definition:

- A view is not precomputed or stored.
- It is computed "on the fly" by executing the query whenever the view is accessed.

Key Functions of a View:

1. **Security & Access Control:** Hide sensitive data (e.g., hide salary from a clerk).
2. **Abstraction:** Present a simplified model (e.g., hide complex joins from end-users).

Creating and Using a View

Scenario: We want a list of CS students only.

```
CREATE VIEW cs_students AS
SELECT name, dept
FROM student
WHERE dept = 'CS';
```

Usage: Users treat it like a normal table:

```
SELECT * FROM cs_students;
```

Virtual Result:

Name	Dept
Alice	CS
Carol	CS

4.2.4 Updatability of Views

Can we `INSERT`, `UPDATE`, or `DELETE` on a view? **"NO"**

The Problem:

- If you insert into a view, the database must insert into the underlying base table.
- If the view hides a NOT NULL column (like ID), the insertion will fail.
- If the view contains aggregates (SUM, COUNT), the insertion is impossible (you cannot update a "Sum").

Conditions for Updatable Views

Most SQL standards (including Postgres) allow updates only on **Simple Views**:

1. The `FROM` clause has only one database relation.
2. The `SELECT` clause contains only attribute names (no expressions, aggregates, or `DISTINCT`).
3. Any attribute not listed in the `SELECT` clause must be nullable or have a default value in the base table.
4. No `GROUP BY` or `HAVING` clauses.

4.2.3 Materialized Views

Definition: Unlike standard views, **Materialized Views** physically store the result of the query.
Use Case: Queries that aggregate large amounts of data (e.g., Total Salary by Department). Materializing avoids re-computing the sum every time.
PostgreSQL Syntax:

```
CREATE MATERIALIZED VIEW dept_stats AS
SELECT dept, COUNT(*) as num_students
FROM student
GROUP BY dept;
```

Maintenance: Data can become stale. You must update it explicitly:

```
REFRESH MATERIALIZED VIEW dept_stats;
```

Materialized View

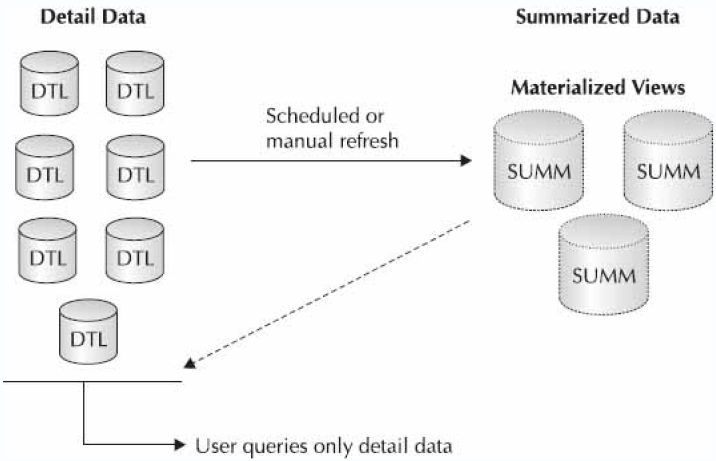


Figure: Materialized View

Transactions

4.3 Transactions: Theory

A **Transaction** is a sequence of query and/or update statements that make up a single logical unit of work.

The ACID Properties (Focus on A):

- **Atomicity:** Either all steps complete, or none do. It is an "All or Nothing" property.
- **Consistency:** The database moves from one valid state to another.

SQL Commands:

- `COMMIT`: Makes updates permanent.
- `ROLLBACK`: Undoes all updates performed by the transaction.

Transaction Example: Bank Transfer

Scenario: Transfer $100 from Account A (ID 1) to Account B (ID 2).

Step 1: Start and Deduct

```
BEGIN;
UPDATE accounts SET balance = balance - 100
WHERE id = 1;
```

System Crash Risk: If power fails here, money is deducted but not added. Atomicity prevents this.

Step 2: Add and Commit

```
UPDATE accounts SET balance = balance + 100
WHERE id = 2;
COMMIT;
```

The `COMMIT` ensures both updates happen together.

Integrity Constraints

4.4 Integrity Constraints: Overview

Constraints are rules that guard against accidental damage to the database. They ensure **Data Consistency**.

Where to enforce?

- *UI Design:* Good for user feedback, but bypassable.
- *Database (Backend):* Essential. Ensures authorized changes do not result in data loss or inconsistency.

Types of Constraints

1. **NOT NULL**: Attribute cannot be null (e.g., Name).
2. **UNIQUE**: Attributes must be unique (Candidate keys).
3. **PRIMARY KEY**: Uniquely identifies a row (Not Null + Unique).
4. **CHECK (P)**: Ensures a predicate P is satisfied.

Example: Check Constraint

Scenario: Salary must be positive, and Department must be one of the allowed values.

```
CREATE TABLE instructors (
id SERIAL PRIMARY KEY,
name VARCHAR(50) NOT NULL,
salary NUMERIC,
dept_name VARCHAR(20),

CONSTRAINT positive_salary CHECK (salary > 0),
CONSTRAINT valid_dept CHECK (dept_name IN ('CS', 'EE', 'Bio'))
);
```

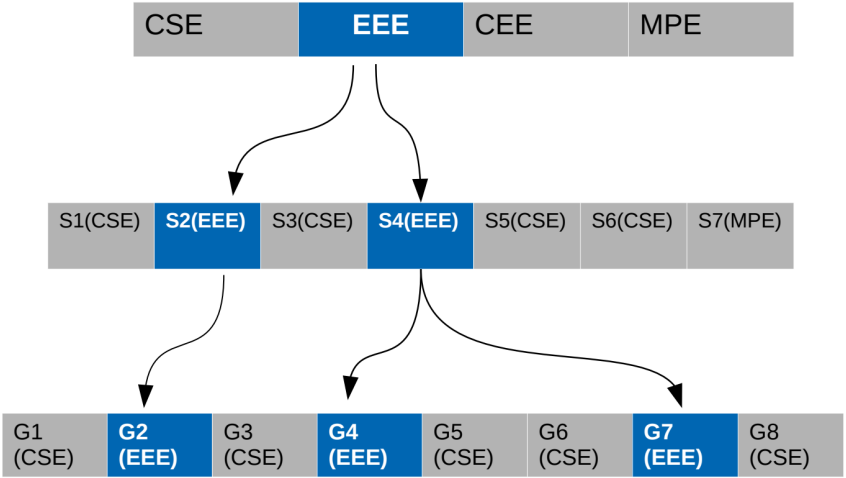
Attempting to insert a negative salary will trigger a database error.

4.4.5 Referential Integrity (Foreign Keys)

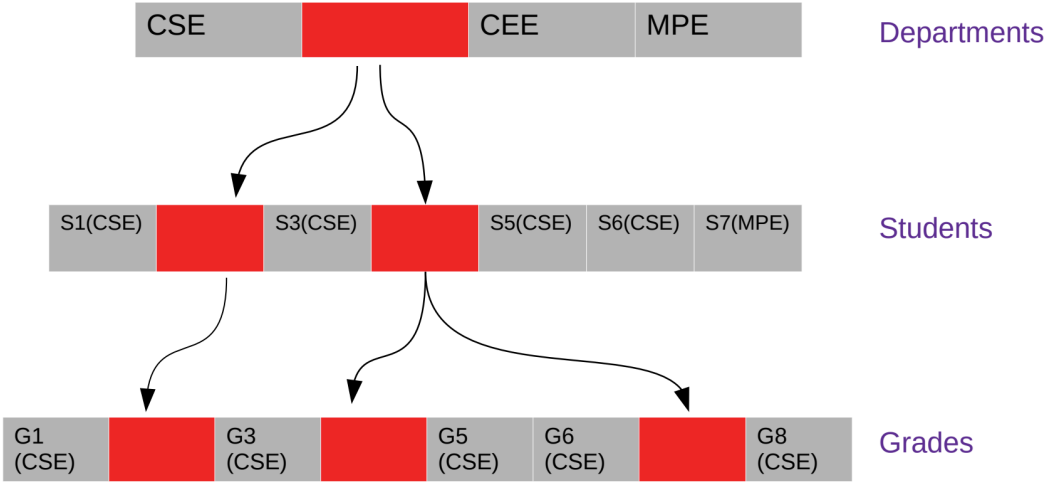
Definition: Ensures that a value appearing in one relation (referencing) also exists in another (referenced).
Cascading Actions: What happens if the referenced data is deleted?

- **ON DELETE NO ACTION:** Default. Rejects the deletion.
- **ON DELETE CASCADE:** Deletes the referencing rows too.
- **ON DELETE SET NULL:** Sets the referencing column to NULL.

Cascading Delete



Cascading Delete



Cascading Delete: Example

Scenario: If a Department is deleted, delete all its Courses automatically.

```
CREATE TABLE course (
course_id VARCHAR(10) PRIMARY KEY,
dept_name VARCHAR(20),

FOREIGN KEY (dept_name)
REFERENCES department(dept_name)
ON DELETE CASCADE
);
```

Operation: `DELETE FROM department WHERE dept_name = 'History';`
Result: The database automatically finds all History courses and deletes them to maintain integrity.

Advanced: Deferrable Constraints

The Circular Dependency Problem:

```sql
CREATE TABLE person (
ID int PRIMARY KEY,
spouse_id int REFERENCES person(ID)
);
```

How do we insert a married couple? We cannot insert the husband because the wife doesn't exist yet, and vice versa.

Solution:

```sql
SET CONSTRAINTS ALL DEFERRED;
-- Insert Husband (with dummy spouse_id)
-- Insert Wife
-- Update Husband with real spouse_id
COMMIT;
```

Deferred constraints are checked only at COMMIT time, allowing temporary inconsistency.

Date and Time

4.5.1 Date and Time Types (Postgres)

SQL standard supports:

- **DATE**: Year, month, day (e.g., '2025-10-25').
- **TIME**: Hours, minutes, seconds.
- **TIMESTAMP**: Date and time combined.

Current Time:

- Oracle: `SYSDATE`
- PostgreSQL: `CURRENT_DATE` or `NOW()`

Date Conversion and Formatting

TO_CHAR(): Formatting a date into a string.

```
SELECT TO_CHAR(NOW(), 'DD-Mon-YYYY HH24:MI');
-- Output: 13-Dec-2025 14:30
```

TO_DATE(): Converting a string to a date.

```
SELECT TO_DATE('05 Jan 2017', 'DD Mon YYYY');
-- Output: 2017-01-05
```

TO_CHAR() Function formats

Format Specifier	Meaning
YYYY	4-digit year
YY	2-digit year
MONTH	Month name (January - December)
MM	Month (1 - 12)
DD	Day (1 - 31)
DY	Abbreviated day (Sun - Sat)
Day	Day (Sunday, Monday)
HH24	Hour (0 - 23)
HH or HH12	Hour (1 - 12)
MI	Minutes (0 - 59)
SS	Seconds (0 - 59)

Date Arithmetic (Intervals)

PostgreSQL uses **Intervals** for date math instead of specific functions like `ADD_MONTHS`.

Examples:

```sql
-- Add 7 days
SELECT CURRENT_DATE + INTERVAL '7 days';

-- Add 1 year and 3 months
SELECT CURRENT_DATE + INTERVAL '1 year 3 months';

-- Calculate Age
SELECT AGE('2025-01-01', '2000-01-01');
-- Output: 25 years
```

4.5.4 Large Objects (LOBs)

Used to store large data items (KB to GB sizes).

- **CLOB (Character Large Object):**
  - Used for large text (book chapters, XML).
  - Postgres uses `TEXT`.
- **BLOB (Binary Large Object):**
  - Used for binary data (images, videos, PDF).
  - Postgres uses `BYTEA` or Large Object (LO).

Efficiency: Queries typically return a **locator** (pointer) rather than the object itself.

Create Table Extensions

4.5.6 Create Table Extensions

Often we need to create a table that has the same structure as an existing one.

Syntax (Create Table As Select - CTAS):

```
CREATE TABLE new_students AS
SELECT ID, NAME, GPA
FROM students;
```

This copies both structure AND data.

Trick: Copy Structure Only

```
CREATE TABLE empty_student_table AS
SELECT * FROM students
WHERE 1=0;
```

Since 1=0 is false, no rows are copied, only columns.

Authorization

4.6 Authorization (RBAC)

Role-Based Access Control (RBAC):

- Used to distribute a large system into a number of users with different access controls.
- Instead of managing users individually, we manage **Roles**.

Privileges:

- `SELECT`, `INSERT`, `UPDATE`, `DELETE` (Data).
- `EXECUTE` (Functions).
- `REFERENCES` (Foreign Keys).

Granting and Revoking

Grant Syntax:

```sql
GRANT SELECT, INSERT
ON students
TO faculty_role;
```

Revoke Syntax:

```sql
REVOKE INSERT
ON students
FROM faculty_role;
```

Roles and Hierarchy

Roles can be assigned to other roles.

Example:

```sql
CREATE ROLE teacher;
CREATE ROLE dean;

-- Give teacher permissions to dean
GRANT teacher TO dean;
```

Transfer of Privileges:

- By default, a user cannot pass their privileges to others.
- **WITH GRANT OPTION**: Allows the recipient to further grant the privilege.

RBAC Example

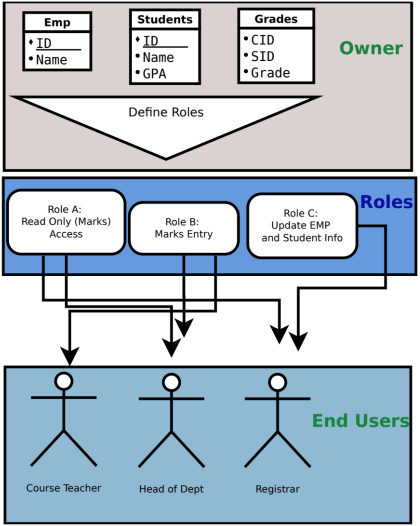


Figure: Role-based Access Control in Result Processing System (RPS)

Summary

- **Joins:** Master Inner, Left, Right, and Full joins to link tables. Avoid Natural Join pitfalls.
- **Views:** Use for abstraction. Understand update limitations.
- **Transactions:** ACID ensures data safety (Commit/Rollback).
- **Constraints:** Essential for backend data integrity (Check, FK, Cascade).
- **Authorization:** Use Roles for scalable security management.

Questions?